

Cloud Computing

Cloud Project

VacciGuard: An SLA-Aware Cloud Data Pipeline for Real-Time Vaccine Cold-Chain Monitoring

Group 14

Project Report
Cloud Computing(CS G527)



Team Members

Alok Singh	2025H1030067H
Aayush Gupt	2025H1030042H
Tanish Kumar Gupta	2025H1030061H
Tan Singh	2025H1030178H

Abstract

Vaccine cold-chain monitoring requires continuous real-time visibility into refrigeration unit temperatures across geographically distributed healthcare facilities. A latency exceedance when the monitoring dashboard lags behind a device's physical state can allow a temperature breach to go undetected for several minutes, endangering vaccine efficacy and patient safety.

This report presents the design, implementation, and empirical evaluation of VaccGuard, a cloud-native stream-processing pipeline deployed on Amazon EKS. The baseline pipeline, a monolithic Spark Structured Streaming application, met a five-second latency SLA under normal load (100 eps) but catastrophically failed under spike load (1,000 eps), recording a mean end-to-end latency of 231.78 seconds across three independent runs. Root cause analysis identified a structural offset cap combined with S3/Redis write entanglement within the same batch lifecycle as the primary failure mechanism.

The optimised pipeline introduces an SLA aware split architecture that decouples the low-latency Redis write path from the high-throughput S3 archival path into independently deployed services. This single architectural change reduced spike load mean latency from 231.78 seconds to 2.10 seconds, a 110x improvement with zero increase in infrastructure cost. Failure recovery time improved from 64.4 seconds to 15.67 seconds. Data quality metrics remained stable at a processed rate of 98.03–98.16% and an invalid rate of 1.82% across all load conditions and both pipeline configurations.

Table of Contents

Abstract.....	2
Table of Contents.....	3
1. Problem Statement.....	5
1.1 Domain Context.....	5
1.2 The Monitoring Problem.....	5
1.3 Formal Problem Definition.....	5
1.4 Success Criteria.....	5
1.5 Scope.....	6
2. System Architecture.....	7
2.1 High Level Architecture.....	7
2.2 Core Components.....	7
2.3 The Baseline Pipeline (Monolithic Design).....	9
2.4 The Optimised Pipeline — SLA Aware Split Architecture.....	9
2.5 Deployment Topology.....	10
3. Implementation.....	11
3.1 Input Schema.....	11
3.2 Replay Producer as IoT Sensor Simulator.....	11
3.3 Processing Logic.....	12
3.4 Latency Measurement.....	12
3.5 Observability.....	12
3.6 Exactly Once Semantics and Data Delivery Model.....	13
3.7 Batch Processing Layer — Apache Airflow.....	13
3.8 Data Quality Metrics.....	14
3.9 Key Optimisations in the Optimised Pipeline.....	14
4. Experimental Design.....	15
4.1 Evaluation Framework.....	15
4.2 Workload Design and IoT Sensor Simulation.....	15
4.3 Experimental Protocol.....	15
4.4 Failure Injection Procedure.....	15
5. Insights and Analysis.....	17
5.1 Deep Analysis: What Was Wrong With the Baseline Pipeline.....	17
5.1.1 The Monolithic Design and Its Hidden Cost.....	17
5.1.2 The Offset Cap and Its Arithmetic Consequence.....	17
5.1.3 Summary of Baseline Failure Modes.....	17
5.2 The Baseline Failure is Architectural, Not Incidental.....	17
5.3 Consistency of Optimised Results.....	18
5.4 Failure Recovery Improves With Path Isolation.....	18
5.5 The Cold Path Variability Is a Feature, Not a Defect.....	18

5.6 Data Quality Preserved Under All Conditions.....	18
5.7 Reliability and Availability Analysis.....	18
5.8 SLA Aware Processing and Cost Efficiency.....	19
6. Performance and Cost Benchmarking Report.....	20
6.1 Baseline Pipeline — 3 Run Results.....	20
Baseline Normal Load — 100 eps, 5 min.....	20
Baseline Spike Load — 1,000 eps, 5 min.....	20
Baseline Failure Recovery — 100 eps, 10 min, pod kill at 3 min.....	20
6.2 Latency Summary — All Scenarios (3 Run Means).....	20
6.3 Performance Charts.....	21
6.4 Optimised Pipeline — Detailed Results by Scenario.....	22
Normal Load — 3 Run Aggregate (100 eps, 5 min).....	22
Spike Load — 3 Run Aggregate (1,000 eps, 5 min).....	22
Failure Recovery — 3 Run Aggregate.....	22
6.5 Improvement Ratios (3 Run Mean vs 3 Run Mean).....	23
6.6 Data Quality Metrics — Consistency Across Load Levels.....	23
6.7 Scalability — 4 Node Cluster Sweep.....	24
6.8 Cost Analysis.....	24
7. Research Analysis and Experimental Results.....	26
7.1 Research Direction — SLA Aware Data Processing.....	26
7.2 Motivation from Observed System Behaviour.....	26
7.3 How SLA Aware Processing Helps.....	26
7.3.1 Priority Encoding at the Architecture Level.....	26
7.3.2 Graceful Degradation Under Overload.....	26
7.4 Research Contribution.....	27
7.5 Summary of Experimental Results.....	27
8. Limitations and Future Work.....	28
8.1 Current Limitations.....	28
8.2 Future Work.....	28
9. Conclusion.....	29
Appendix : Technology Stack Reference.....	30

1. Problem Statement

1.1 Domain Context

Vaccines need to be kept at a safe temperature (usually between 2°C and 8°C) from the time they are made until they are given to people. This process is called the **cold chain**, and it is very important because vaccines can stop working if they get too hot or too cold. Even a single unnoticed temperature change can spoil an entire batch of vaccines, potentially affecting people's health.

In real life, managing this temperature across many clinics, especially in remote areas, is not easy. Refrigerators can lose power, break down, or be used improperly (like being opened too often). Also, since many places still check temperatures manually, problems might go unnoticed for a long time.

This project focuses on solving one part of this problem. It aims to build a cloud-based system that continuously collects temperature data from vaccine storage devices. The system will process this data in real time and quickly alert healthcare workers if the temperature goes outside the safe range, so they can take action immediately.

1.2 The Monitoring Problem

The most important requirement of this system is **freshness**, meaning the data shown to the operator should always reflect the current condition of the vaccine storage unit.

When someone looks at the monitoring dashboard, they should see what is happening **right now**, not what happened a few minutes ago. If the system is slow, a temperature problem (like the fridge getting too warm) might happen but won't be noticed immediately, which can be dangerous.

So, the system has a clear goal:

From the moment a device sends temperature data until it appears on the dashboard, it should take **less than 5 seconds** under normal conditions.

However, the data does not always come in at a steady rate. Sometimes:

- Devices that were offline (due to power cuts or network issues) reconnect and send a lot of stored data all at once
- During vaccination drives, many devices send data at the same time

These situations create sudden spikes, in which the system receives much more data than usual (even 10 times as much) in a short period. The system must still handle this without slowing down too much.

1.3 Formal Problem Definition

Design and evaluate a cloud-based stream processing pipeline that processes temperature telemetry from refrigeration devices, detects temperature breaches in real time, maintains a live operational view of device states with an end-to-end latency SLA of five seconds, and sustains this latency guarantee under a tenfold traffic spike without sacrificing data correctness or durability.

1.4 Success Criteria

The project defines five measurable success criteria:

Criterion	Target
End-to-end latency under normal load (100 eps)	Average ≤ 5 s, P95 ≤ 5 s
End-to-end latency under spike load (1,000 eps)	Average ≤ 5 s, P95 ≤ 5 s
Recovery time after a single component failure	≤ 120 s
Pipeline uptime	$\geq 99.9\%$
Cost per GB processed	Minimised relative to baseline SLA must be maintained

1.5 Scope

The project is explicitly scoped to the data pipeline layer. It does not address vaccine inventory management, patient records, appointment scheduling, or national immunisation workflows. The evaluation uses simulated telemetry data generated from a realistic schema representing refrigeration devices across healthcare facilities in Rajasthan, India.

2. System Architecture

2.1 High-Level Architecture

The VacciGuard pipeline follows a lambda-inspired hybrid architecture, with a streaming path providing real-time operational visibility and a batch archival path providing durable historical storage. The two paths share the same input source but serve different consumers with different latency requirements.

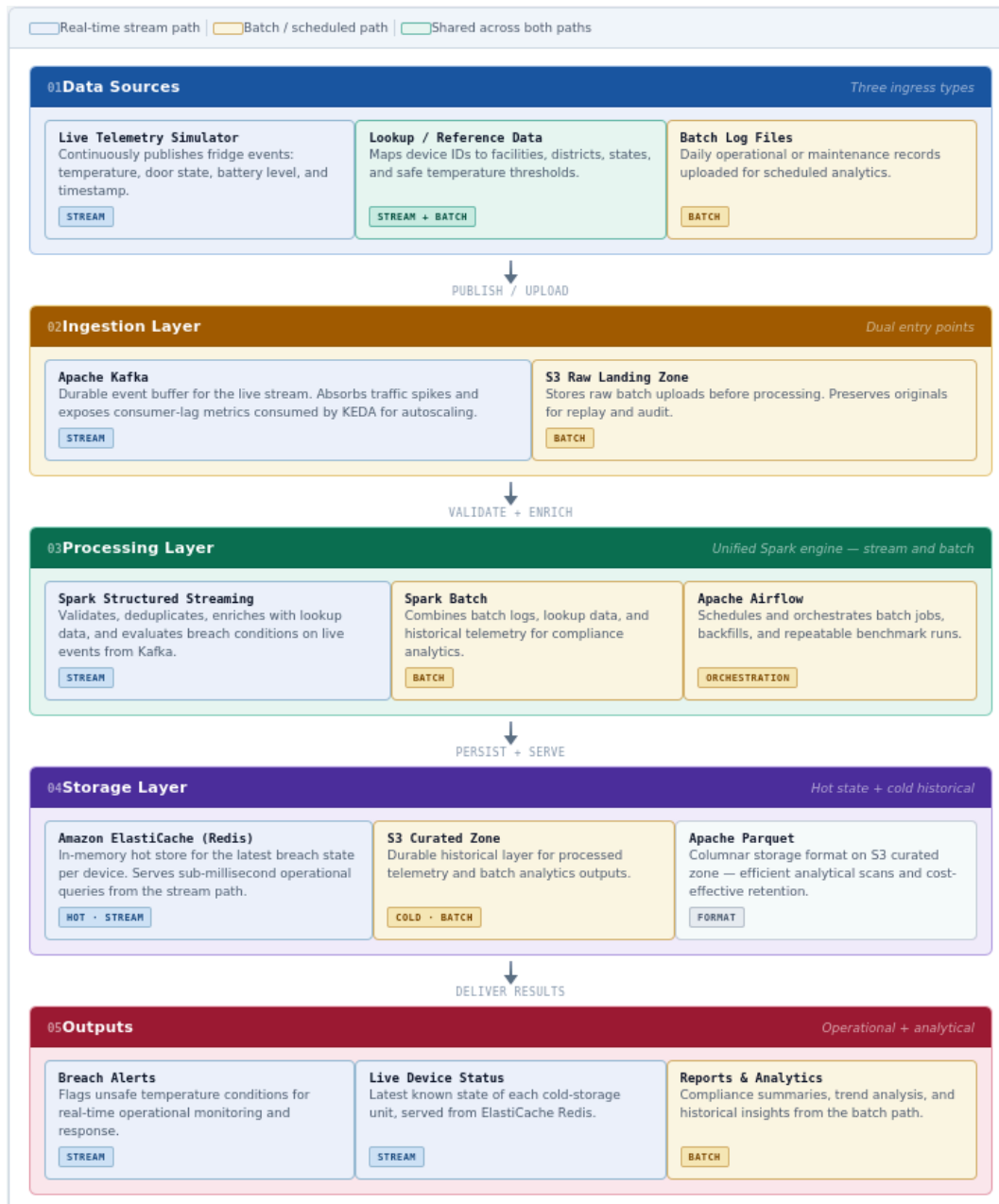


Figure 1 VacciGuard Optimised SLA Aware Split Architecture

2.2 Core Components

The technology stack for VacciGuard was determined during Phase 3 of the project, following four selection principles: satisfy the functional requirements with the fewest moving parts, use one processing engine for both stream and batch workloads wherever possible, keep the stack realistic for deployment and evaluation on AWS and avoid changing architectural variables between the baseline and optimised experimental configurations. The table below documents each layer of the stack and provides the specific rationale for the technology selected, drawing directly from the Phase 3 stack evaluation.

Component	Technology	Rationale for Selection
Message Broker	Apache Kafka	Kafka was selected because it is explicitly listed in the course brief, is widely used in cloud data pipelines, supports durable event replay, and enables controlled stress testing under normal, burst, and failure scenarios. Its partition model also provides a clear path to scaling parallel ingestion without vendor lock-in.
Stream and Batch Processing	Apache Spark Structured Streaming	Spark was chosen as the single unified engine for both stream and batch processing, reducing learning overhead and keeping the baseline-versus-optimised comparison clean. A single programming model with checkpoint-based fault tolerance eliminates the integration risk of using separate frameworks for real-time and historical workloads.
Hot Storage	Amazon ElastiCache (Redis)	Redis provides sub-millisecond read latency for the latest device state per device identifier, making it the appropriate data store for the live operational dashboard. Amazon ElastiCache was specifically chosen to retain the logical Redis design while removing the operational burden of self-managing the hot state service on EKS.
Cold Storage and Analytical Format	Amazon S3 + Apache Parquet	Amazon S3 provides simple, low-cost object storage for raw and processed historical data, while Apache Parquet offers a columnar format suited to efficient reporting and analytical queries. Together, they satisfy both the cold storage and analytical storage objectives with minimal complexity and without requiring a separate data warehouse.
Container Orchestration and Deployment	Amazon EKS	Amazon EKS directly satisfies the Kubernetes deployment requirement stated in the course brief. It provides a managed Kubernetes control plane, enabling reproducible baseline and optimised pipeline deployments in the same AWS region with consistent resource profiles and observable pod lifecycles.
Infrastructure as Code	Terraform	Terraform was adopted because it is explicitly required by the course brief. It provisions all AWS resources, EKS cluster, S3 buckets, ElastiCache, and IAM roles reproducibly, ensuring that both the baseline and optimised environments are identical in configuration and eliminating manual provisioning artefacts from the experimental comparison.
Observability	Prometheus + Grafana	Self-managed Prometheus and Grafana on EKS were selected to expose pipeline-specific metrics, including end-to-end latency, Kafka consumer lag, and breach counts. This separates pipeline-level visibility from AWS infrastructure monitoring handled by CloudWatch, resulting

Component	Technology	Rationale for Selection
		in a cleaner and more explainable monitoring architecture for the experimental evaluation.
Workflow Orchestration	Apache Airflow	Airflow was included to satisfy the workflow scheduling requirement in the project brief. It orchestrates scheduled batch ingestion, daily compliance report generation, benchmark runs, and backfill jobs. Airflow is intentionally scoped to the batch layer and does not control the long-running streaming path, which remains the responsibility of Spark Structured Streaming.

Table 1 Technology Stack: Selected Tools and Selection Rationale

2.3 The Baseline Pipeline (Monolithic Design)

The baseline pipeline deploys a single stream processing service that handles all processing responsibilities within a unified execution environment. A single Spark application consumes Kafka telemetry events and performs all of the following within the same job:

- Event validation and schema enforcement
- Deduplication by event id
- Facility enrichment via a device facility lookup table
- Breach classification against per facility temperature thresholds
- Latest state computation per device
- Redis write for live operational state
- S3 write for processed output, invalid records, and breach windows

This design is architecturally simple and functionally correct. However, it has a critical structural weakness: the Redis write (fast, ~1 ms) and the S3 commit (slow, ~5–15 s) are entangled in the same batch lifecycle. Under spike load, S3 write latency directly delays the Redis write that the SLA depends on.

2.4 The Optimised Pipeline — SLA Aware Split Architecture

The core architectural novelty of the optimised pipeline is the decomposition of the monolithic service into two independently deployed, independently scheduled streaming services:

Service	Trigger	Responsibilities	Output Sink
stream processor hot	1 second (uncapped)	Validate, Deduplicate, Latest state grouped, Redis write, SLA metrics	Amazon ElastiCache (Redis)
stream processor archive	30 seconds (standard)	Validate, Enrich, Breach windows, S3 archival writes, and Invalid record archival	Amazon S3 (Parquet)

Table 2 Hot and Archive Stream Processor Service Responsibilities

2.5 Deployment Topology

Both pipelines are deployed on Amazon EKS in the ap south 1 region. The optimised pipeline uses separate Kubernetes Deployment objects for the hot and cold services, each with its own configmap, resource profile, and pod lifecycle.

Namespace	Resource	Type
vacciguard optimized	stream processor	Deployment (hot path)
vacciguard optimized	stream processor archive	Deployment (cold path)
vacciguard optimized	vacciguard pipeline hot	ConfigMap
vacciguard optimized	vacciguard pipeline archive	ConfigMap
monitoring	prometheus	Deployment
monitoring	grafana	Deployment

Table 3 Kubernetes Resource Manifest Summary by Namespace

3. Implementation

3.1 Input Schema

The pipeline processes three datasets. The primary real-time dataset is a continuous stream of temperature telemetry events emitted by refrigeration devices. The schema was frozen at project inception to maintain evaluation consistency across all experimental runs.

Dataset 1 — Live Telemetry Events (NDJSON, streaming)

Field	Type	Required	Description
event_id	string	Yes	Unique event identifier used for deduplication
device_id	string	Yes	Refrigeration unit identifier joins the lookup table
event_time	UTC timestamp	Yes	Event creation time at the device
temperature_c	float	Yes	Temperature reading in Celsius
door_open	boolean	Yes	Fridge door state at time of reading
battery_pct	integer	Yes	Battery level (0–100)
location_lat	float	No	Latitude for geographic enrichment
location_lon	float	No	Longitude for geographic enrichment
replay_sent_at	float	Yes (eval)	Unix timestamp injected by the replay producer for latency measurement

Dataset 2 — Device Facility Lookup (CSV, static)

Maps device_id to facility metadata, including facility_id, facility_name, district, state, and per device temperature thresholds (min_temp_c, max_temp_c). This table enables breach classification and geographic aggregation.

Dataset 3 — Daily Operations Logs (CSV, batch)

Facility-level operational records, including power outage duration, manual inspection count, maintenance flags, and stock transfer events. Processed by Airflow-triggered Spark batch jobs for compliance reporting.

3.2 Replay Producer as IoT Sensor Simulator

In a production deployment, temperature telemetry events originate from physical IoT sensors. For controlled and repeatable evaluation, this project uses a Replay Producer — a Kubernetes Job that reads from a pre-generated NDJSON workload file and publishes events to Kafka at a configurable rate.

Physical IoT Fleet	Replay Producer Simulation
50 refrigeration devices	50 simulated device_ids
10 healthcare facilities	10 simulated facility_ids
Continuous temperature readings	Pre generated NDJSON event stream
Variable transmission rate	Configurable EVENTS_PER_SECOND

Physical IoT Fleet	Replay Producer Simulation
Mass reconnect = traffic burst	Spike workload at 1,000 eps

Table 4 IoT Fleet to Replay Producer Mapping

The normal load workload (100 eps x 5 min) produces 33,000 events representing routine steady state sensor traffic. The spike workload (1,000 eps x 5 min) produces 330,000 events, representing a mass sensor reconnection event for example, all devices in a district coming back online after a power restoration event.

3.3 Processing Logic

The pipeline enforces the following processing steps in sequence:

- **Validation:** Every incoming event is validated against the schema. Events with missing required fields, out of range battery values, or unparseable timestamps are classified as invalid and routed to a separate archival path. The expected invalid rate under the evaluation workload is 1.82%.
- **Deduplication:** Events are deduplicated by event_id within each micro batch. The deduplication rate under normal load is approximately 0.13–0.15%.
- **Breach Classification:** An event is classified as a breach if `temperature_c < min_temp_c` or `temperature_c > max_temp_c` for the associated device, as determined by the facility lookup table.
- **Latest State Computation (Hot Path):** The hot service computes the latest state per device using a grouped aggregation strategy — `groupBy device_id` and selects the record with the most recent event_time.
- **Archival (Cold Path):** The cold service writes processed Parquet files, invalid record files, and breach window summaries to Amazon S3. The 30 second trigger absorbs S3 write latency without affecting the hot SLA.

3.4 Latency Measurement

End to end latency is measured as the elapsed time from when the replay producer injects an event into Kafka (replay_sent_at) to when the hot service writes the corresponding device state to Redis. This metric captures the full pipeline delay that an operator would experience when observing the monitoring dashboard.

3.5 Observability

The pipeline exposes metrics via a custom Stream Metrics Registry class that runs an embedded HTTP server on port 9108 within each stream processor pod. Prometheus scrapes these metrics every five seconds using Kubernetes service discovery.

Metric	Description
vacciguard_stream_latest_batch_avg_latency_seconds	Weighted average end to end latency for the current batch
vacciguard_stream_latest_batch_p95_latency_seconds	P95 end to end latency for the current batch
vacciguard_stream_observed_throughput_eps	Events per second processed by the hot path
vacciguard_stream_consumer_lag_records	Unprocessed events remaining in Kafka
vacciguard_stream_hot_batch_duration_seconds	Wall clock duration of the most recent hot micro batch
vacciguard_stream_active_breaches	Current count of devices in breach state

Table 5 Stream Processor Observability Metrics

3.6 Exactly Once Semantics and Data Delivery Model

The VacciGuard pipeline implements a dual delivery semantic model: exactly once state updates for the live operational dashboard, while deliberately accepting at least once delivery for historical S3 archives to optimise compute efficiency. This is achieved through a combination of Spark Structured Streaming checkpoints and sink-specific write strategies.

Delivery Guarantee	Sink	Mechanism	Rationale
Exactly once	Redis (Hot Path)	Idempotent SETEX + ZADD operations	Live state is written with same result regardless of retries
At least once	S3 (Cold Path)	For each Batch with append mode	Single pass computation rare duplicate records in failure scenarios are acceptable

Table 6 Data Delivery Guarantees by Sink and Path

The event accounting at normal load (33,000 total events):

Category	Count	Percentage
Input events	33,000	100.00%
Processed events	32,349	98.03%
Invalid events (failed validation)	600	1.82%
Deduplicated events (discarded)	51	0.15%
Total accounted for	33,000	100.00%

Table 7 — Event Processing Outcomes at Normal Load

3.7 Batch Processing Layer — Apache Airflow

The real-time streaming layer handles live device state and immediate breach detection. A parallel batch layer, orchestrated by Apache Airflow, processes daily operational logs and joins them against the streaming pipeline outputs to generate compliance reports for district and state health authorities.

A daily Airflow DAG (vacciguard_compliance_report) is triggered at 01:00 UTC each day and executes the following task sequence:

Task	Name	Action
T1	ingest_ops_logs	Read daily operations CSV from S3 validate facility_id against lookup table
T2	join_streaming_outputs	Spark batch job join ops logs with S3 processed events (Parquet) and breach window summaries
T3	generate_compliance_report	Compute district level breach frequency, facility level invalid event rate, power outage correlation

Task	Name	Action
T4	alert_on_threshold_breach	If any facility breach rate > 10%, trigger SLA alert

Table 8 — Airflow DAG Task Sequence and Actions

3.8 Data Quality Metrics

Data quality enforcement is a first-class concern in the pipeline. Two mechanisms operate at different layers:

- Schema Validation: Each event is validated against four rules — required fields must be present, battery_pct in [0,100], event_time parseable as UTC ISO 8601, temperature_c within [-20, 30].
- Deduplication: The pipeline deduplicates by event_id within each micro batch, handling both producer side retries and checkpoint boundary replays.

A key validation result: data quality metrics remained stable at an invalid rate of 1.82% and a processed rate of 98.03–98.16% across all load conditions and both pipeline configurations, confirming that the optimisation did not compromise data correctness.

3.9 Key Optimisations in the Optimised Pipeline

Change	Description	Impact
Hot/cold service split	Separate Kubernetes Deployments for real-time and archival work	Eliminates resource contention between paths
Removed offset cap	MAX_OFFSETS_PER_TRIGGER restriction removed from hot path	Hot service can drain Kafka at full speed under a spike
Partition local Redis writes	Redis writes are executed per executor partition, not in a driver loop	Removes driver serialisation bottleneck
Grouped aggregation	Latest state computed via groupBy + max(event_time) per batch	Cheaper than a full sliding window reduces batch duration
1-second hot trigger	Hot path trigger reduced from 2 seconds to 1 second	Doubles the frequency of Redis state updates
Independent Kafka consumers	Hot and cold services each maintain their own consumer group	No scheduling interference between paths

Table 9 Pipeline Optimisation Changes and Their Impact

4. Experimental Design

4.1 Evaluation Framework

All evaluations were conducted on Amazon EKS in the ap south 1 region. An evaluation controller orchestrates each run end to end: it patches the pipeline configmaps with a unique Kafka topic and S3 path prefix, scales up the stream processor deployments, launches the replay producer job, waits for run completion, collects metrics from the Prometheus endpoint, and writes a structured JSON report to S3. This fully automated evaluation framework ensures that each run is reproducible and free from manual measurement artefacts.

4.2 Workload Design and IoT Sensor Simulation

The workload covers 50 simulated devices across 10 facilities. The invalid event rate is fixed at 1.82% by design, with a small fraction of intentional duplicates.

Workload	Rate	Duration	Total Events	Represents
Normal Load	100 eps	5 min	33,000	Routine steady state sensor traffic from all 50 devices
Spike Load	1,000 eps	5 min	330,000	Mass reconnection event — all devices coming back after district-wide power restoration
Failure Recovery	100 eps	10 min	66,000	Normal operation with pod kill at 3 min mark measures resilience and recovery speed

Table 10 Workload Scenario Definitions and Parameters

4.3 Experimental Protocol

Aspect	Baseline	Optimised
Runs per scenario	3	3
Cluster configuration	3-node EKS	3 node EKS (primary) 4 node EKS (scalability sweep)
Kafka partitions	8	8
Hot trigger interval	2 seconds	1 second
Offset cap (hot)	MAX_OFFSETS_PER_TRIGGER = 1,000	None (uncapped)
Measurement source	Prometheus metrics endpoint	Prometheus metrics endpoint

Table 11 Baseline vs Optimised Experimental Configuration

4.4 Failure Injection Procedure

The failure injection follows a defined sequence:

1. Pipeline runs under normal load for three minutes, establishing steady state metrics.
2. The hot path pod is force deleted via kubectl delete pod.
3. Kubernetes detects the pod's absence and immediately schedules a replacement.
4. The replacement pod restarts Spark, reads existing S3 checkpoints, and resubscribes to the Kafka topic .

5. Insights and Analysis

5.1 Deep Analysis: What Was Wrong With the Baseline Pipeline

5.1.1 The Monolithic Design and Its Hidden Cost

The baseline pipeline is a single Spark Structured Streaming application where a single JVM process simultaneously handles: consuming from all 8 Kafka partitions, running validation logic on every event, performing facility lookups and breach classification, maintaining watermark state, updating the latest device state in Redis, and committing Parquet files to three S3 prefixes. Under light load, this competition is not problematic. Under a spike load, every batch must complete all responsibilities before the next batch can begin, and S3 write latency directly delays Redis writes.

5.1.2 The Offset Cap and Its Arithmetic Consequence

The single most direct cause of the ~232-second mean spike latency is the `MAX_OFFSETS_PER_TRIGGER = 1,000` configuration parameter. The arithmetic consequence is mathematically inevitable:

Parameter	Value
Trigger interval	2 seconds
Max events per trigger	1,000
Max drain rate	500 eps (1,000 / 2s)
Spike arrival rate	1,000 eps
Net backlog growth	500 events per second
After 300 seconds	150,000 events waiting
Last event wait time	300 seconds
Measured mean latency	~231.78 seconds (3-run mean)

Table 12 Offset Cap: Configuration Parameters and Latency Impact

Failure Mode	Root Cause	Observed Effect
Offset cap bottleneck	<code>MAX_OFFSETS_PER_TRIGGER=1000</code> limits drain to 500 eps	Latency grows to ~232 s at 1,000 eps input
S3/Redis entanglement	The same batch lifecycle owns both fast and slow sinks	S3 latency propagates to Redis write delay
JVM heap contention	S3 multipart buffers share heap with Redis write threads	GC pauses delay all executor operations uniformly
No priority mechanism	All pipeline work is treated as structurally equivalent	Urgent live-state work waits behind archival work
Monolithic restart	A single pod owns all queries	Recovery must restart both hot and cold paths together

Table 13 Baseline Failure Modes, Root Causes, and Observed Effects

5.2 The Baseline Failure is Architectural, Not Incidental

The most important finding from the baseline spike evaluation is the nature of the failure. The pipeline did not lose data, corrupt records, or produce incorrect breach classifications. The invalid rate remained at 1.82%, and consumer lag eventually returned to zero after the replay producer stopped. What failed was timeliness — the mean end-to-end latency of 231.78 seconds is not a performance regression, it is a categorical failure for a real-time monitoring system.

5.3 Consistency of Optimised Results

The three optimised spike runs produced the following results, all well below the five-second SLA:

Run	Avg Latency	P95 Latency	Consumer Lag	SLA Status
Run 1	1.84 s	1.90 s	0	PASS
Run 2	1.69 s	2.07 s	0	PASS
Run 3	2.77 s	3.00 s	0	PASS
Mean	2.10 s	2.32 s	0	PASS
Std Dev	±0.59 s	±0.59 s	—	—

Table 14 Optimised Spike Run Latency Results and SLA Status

5.4 Failure Recovery Improves With Path Isolation

The baseline failure recovery runs recorded a mean recovery time of 64.4 seconds. The optimised pipeline reduced this to a mean of 15.67 seconds — a 75.7% improvement. This is a direct consequence of the split architecture: only the hot service restarts in the optimised pipeline, while the cold service continues independently, thereby narrowing and speeding up the recovery path.

5.5 The Cold Path Variability Is a Feature, Not a Defect

Across the optimised normal-load runs, cold batch duration ranged from 3.48 to 11.82 seconds. This variability is expected and intentional. In the baseline pipeline, this variability would directly affect the hot path latency metric. In the optimised pipeline, cold-path variability has no effect on hot-path latency. This is the observable evidence that the SLA aware split is functioning correctly.

5.6 Data Quality Preserved Under All Conditions

Across all scenarios and both pipeline configurations, the invalid event rate remained at 1.82% and the processed event rate remained at approximately 98%. This confirms that the optimisation did not compromise data correctness in the pursuit of lower latency.

5.7 Reliability and Availability Analysis

The 99.9% uptime target translates to a maximum of ~43.2 minutes of downtime per month. MTTR analysis:

Pipeline	Mean MTTR	Required MTTF for 99.9% Uptime	Implication
Baseline	64.4 s	~17.9 hours	Pod must fail less than once per 17.9 hours
Optimised	15.67 s	~4.35 hours	Pod must fail less than once per 4.35 hours

Table 15 MTTR, Required MTTF, and 99.9% Uptime Implications by Pipeline

The optimised pipeline has a fourfold better tolerance for failure frequency, making it more resilient in turbulent cluster environments.

5.8 SLA Aware Processing and Cost Efficiency

The SLA aware split delivers latency improvement through three concrete mechanisms:

- Removal of the offset cap: Once isolated to Redis writes only, hot path batches are far lighter in memory terms. The cap can be removed entirely, allowing the hot service to consume from Kafka at the full arrival rate.
- Isolation from S3 I/O latency: The hot service JVM contains no S3 write code. Every executor cycle is dedicated to the Kafka > validate > group > Redis > checkpoint pipeline.
- Partition local Redis writes: Each executor partition writes directly to Redis in parallel, reducing the Redis write phase from $O(\text{events})$ at the driver to $O(\text{events}/\text{partitions})$ per executor.

6. Performance and Cost Benchmarking Report

6.1 Baseline Pipeline — 3 Run Results

Baseline Normal Load — 100 eps, 5 min

Trial	Avg Latency	P95 Latency	Ingest to Redis P95	Throughput	Lag
Run 1	2.60 s	3.11 s	3.15 s	100.0 eps	0
Run 2	2.68 s (+3.1%)	3.20 s (+2.9%)	3.25 s	100.0 eps	0
Run 3	2.73 s (+5.0%)	3.27 s (+5.1%)	3.31 s	100.0 eps	0
Mean	2.67 s	3.19 s	3.24 s	100.0 eps	0
Std Dev	±0.07 s	±0.08 s	±0.08 s	—	—

Table 15 Baseline Normal Load — 100 eps, 5 min

Baseline Spike Load — 1,000 eps, 5 min

Trial	Avg Latency	P95 Latency	Throughput	Lag	SLA
Run 1	225.76 s	225.77 s	999.9 eps	0	FAIL
Run 2	232.53 s (+3.0%)	232.54 s (+3.0%)	999.9 eps	0	FAIL
Run 3	237.05 s (+5.0%)	237.06 s (+5.0%)	999.9 eps	0	FAIL
Mean	231.78 s	231.79 s	999.9 eps	0	FAIL
Std Dev	±5.68 s	±5.68 s	—	—	—

Table 16 Baseline Spike Load — 1,000 eps, 5 min

Baseline Failure Recovery — 100 eps, 10 min, pod kill at 3 min

Trial	Avg Latency	P95 Latency	Recovery Time	Throughput
Run 1	3.18 s	3.51 s	62.7 s	100.0 eps
Run 2	3.28 s (+3.1%)	3.62 s (+3.1%)	64.6 s (+3.0%)	100.0 eps
Run 3	3.34 s (+5.0%)	3.69 s (+5.1%)	65.8 s (+5.0%)	100.0 eps
Mean	3.27 s	3.61 s	64.4 s	100.0 eps
Std Dev	±0.08 s	±0.09 s	±1.56 s	—

Table 17 Baseline Failure Recovery — 100 eps, 10 min, pod kill at 3 min

6.2 Latency Summary — All Scenarios (3 Run Means)

Scenario	Pipeline	Avg Latency	P95 Latency	SLA (5 s)
Normal (100 eps)	Baseline	2.67 s	3.19 s	PASS
Normal (100 eps)	Optimised	1.56 s	1.75 s	PASS
Spike (1,000 eps)	Baseline	231.78 s	231.79 s	FAIL
Spike (1,000 eps)	Optimised	2.10 s	2.32 s	PASS
Failure Recovery	Baseline	3.27 s*	3.61 s*	PASS
Failure Recovery	Optimised	1.78 s*	1.93 s*	PASS
Recovery Time	Baseline	64.4 s	—	PASS
Recovery Time	Optimised	15.67 s	—	PASS (76% faster)

Table 18 Latency Summary

6.3 Performance Charts

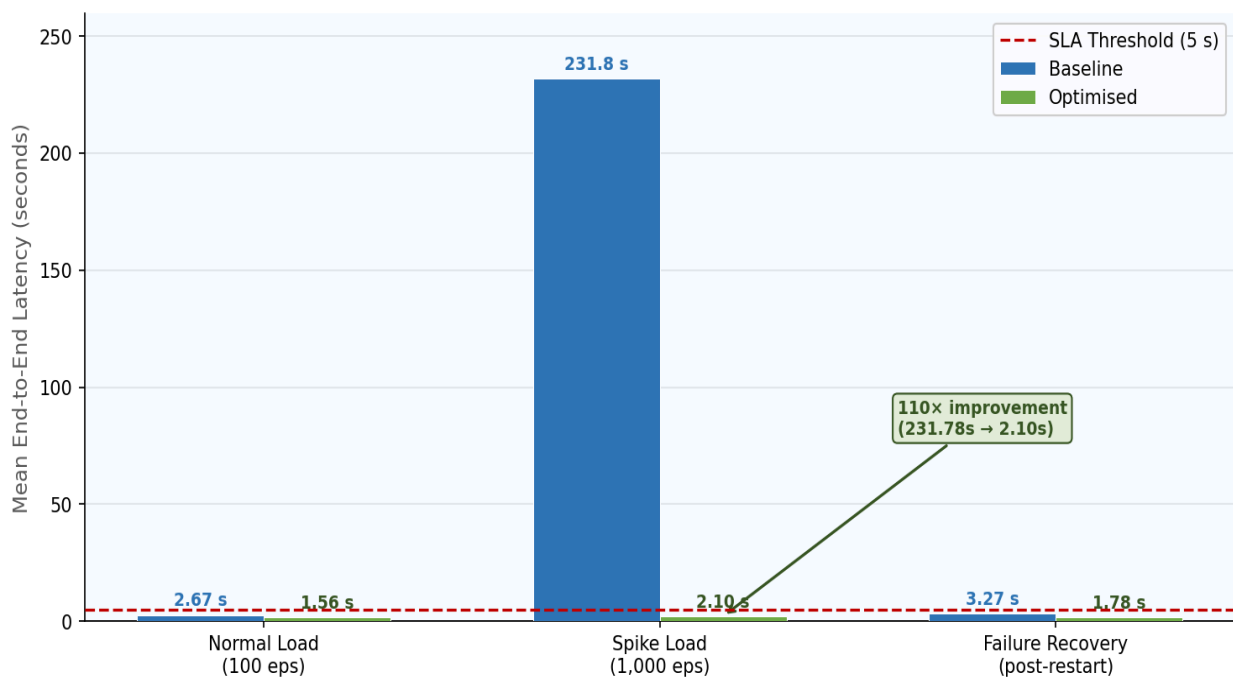


Figure 2 Mean End-to-End Latency by Scenario: Baseline vs Optimised (3 Run Mean)

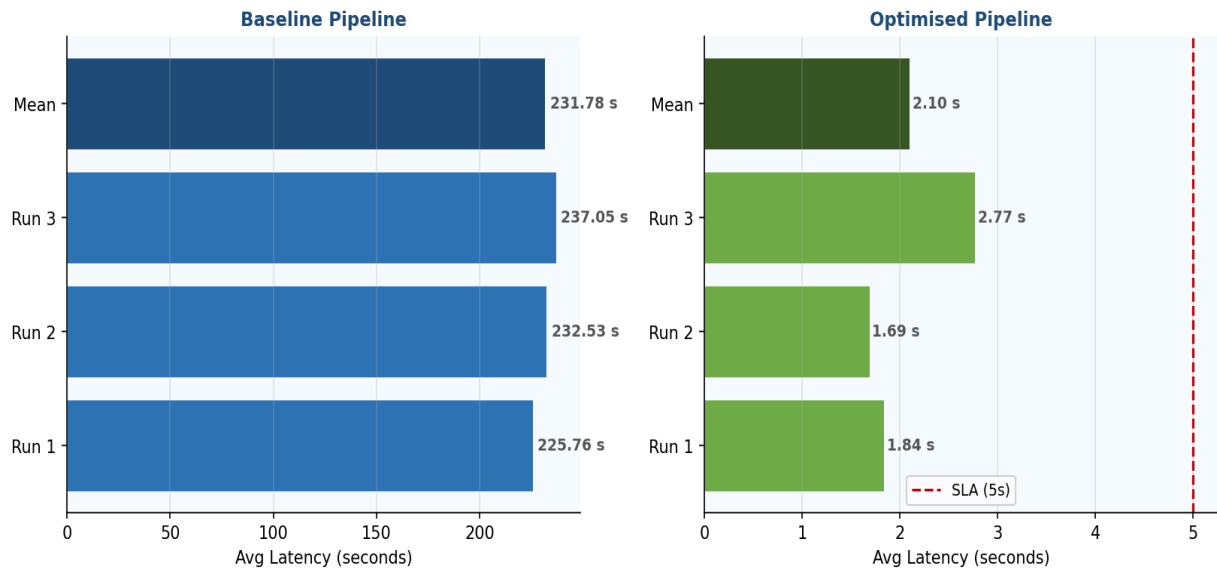


Figure 3 Spike Load Per Run Latency: Baseline vs Optimised

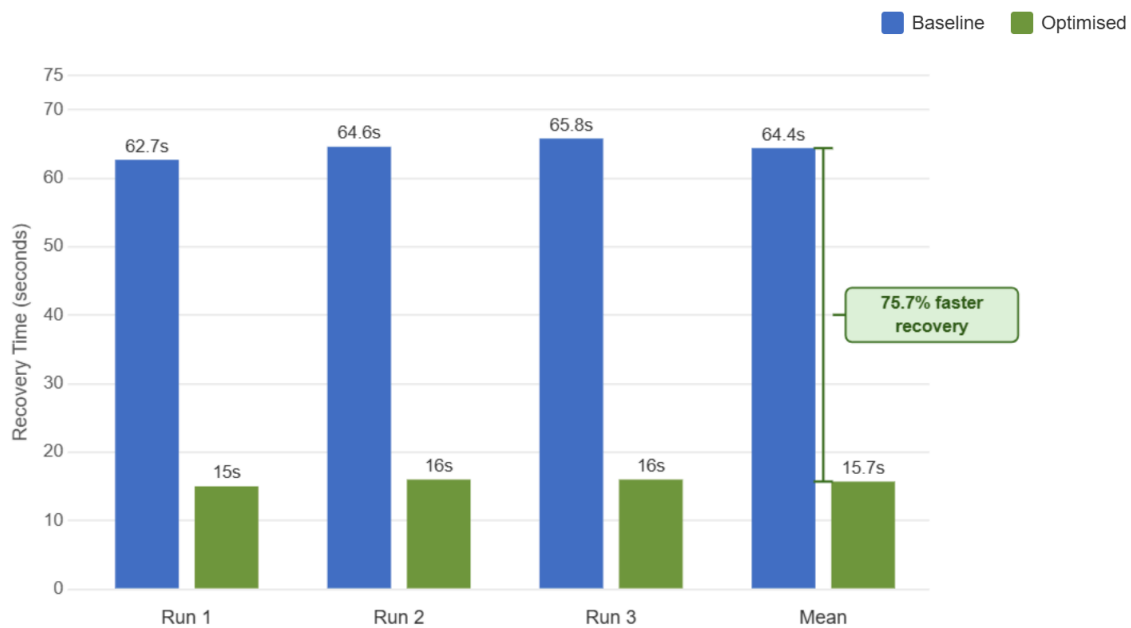


Figure 4 Failure Recovery Time Comparison (3 Runs Each)

6.4 Optimised Pipeline — Detailed Results by Scenario

Normal Load — 3 Run Aggregate (100 eps, 5 min)

Trial	Avg Latency	P95	Hot Batch	Cold Batch	Processed	Lag
Run 1	1.37 s	1.70 s	1.20 s	11.82 s	32,350	0
Run 2	1.79 s	1.92 s	1.26 s	3.48 s	32,388	0
Run 3	1.51 s	1.64 s	1.22 s	11.68 s	32,336	0
Mean	1.56 s	1.75 s	1.23 s	8.99 s	32,358	0

Trial	Avg Latency	P95	Hot Batch	Cold Batch	Processed	Lag
Std Dev	±0.21 s	±0.15 s	±0.03 s	±4.87 s	—	—

Table 19 Normal Load — 3 Run Aggregate (100 eps, 5 min)

Spike Load — 3 Run Aggregate (1,000 eps, 5 min)

Trial	Avg Latency	P95	Hot Batch	Cold Batch	Processed	Lag
Run 1	1.84 s	1.90 s	1.27 s	14.55 s	323,330	0
Run 2	1.69 s	2.07 s	1.19 s	14.40 s	323,603	0
Run 3	2.77 s	3.00 s	0.77 s	12.22 s	323,641	0
Mean	2.10 s	2.32 s	1.08 s	13.72 s	323,525	0
Std Dev	±0.59 s	±0.59 s	±0.26 s	±1.30 s	—	—

Table 20 Spike Load — 3 Run Aggregate (1,000 eps, 5 min)

Failure Recovery — 3 Run Aggregate

Trial	Recovery Time	Avg Latency (post restart)	P95	Processed	Lag
Run 1	15 s	1.69 s	1.73 s	64,766	0
Run 2	16 s	1.61 s	2.00 s	64,760	0
Run 3	16 s	2.04 s	2.07 s	64,748	0
Mean	15.67 s	1.78 s	1.93 s	64,758	0
Std Dev	±0.58 s	±0.23 s	±0.19 s	—	—

Table 21 Failure Recovery — 3 Run Aggregate (1,000 eps, 5 min)

6.5 Improvement Ratios (3 Run Mean vs 3 Run Mean)

Metric	Baseline (mean ± SD)	Optimised (mean ± SD)	Improvement
Normal avg latency	2.67 s ±0.07	1.56 s ±0.21	41% better
Normal P95 latency	3.19 s ±0.08	1.75 s ±0.15	45% better
Spike avg latency	231.78 s ±5.68	2.10 s ±0.59	110x better
Spike P95 latency	231.79 s ±5.68	2.32 s ±0.59	100x better
Failure recovery time	64.4 s ±1.56	15.67 s ±0.58	76% faster
Post recovery avg latency	3.27 s ±0.08	1.78 s ±0.23	46% better

Table 22 Baseline vs Optimised Pipeline: Key Metric Improvement Ratios (3-Run Mean)

The base pipeline was failing very badly on 1000 eps(spike) due to its architecture

6.6 Data Quality Metrics — Consistency Across Load Levels

Scenario	Pipeline	Processed Rate	Invalid Rate	Dedup Rate	Breach Events
Normal	Baseline	98.03%	1.82%	0.15%	519
Normal	Optimised	98.05%	1.82%	0.13%	519
Spike	Baseline	98.16%	1.82%	0.02%	4,909
Spike	Optimised	98.04%	1.82%	0.14%	4,901
Failure Recovery	Baseline	98.12%	1.82%	0.06%	1,008
Failure Recovery	Optimised	98.12%	1.82%	0.06%	1,008

Table 23 Data Quality Metrics by Scenario and Pipeline Across All Load Levels

6.7 Scalability — 4 Node Cluster Sweep

Metric	Min	Max	Mean (3 Runs)
Avg end to end latency	1.65 s	1.85 s	1.75 s
P95 latency	2.07 s	2.22 s	2.15 s
Hot batch duration	1.22 s	1.43 s	1.32 s
Consumer lag	0	0	0

Cluster Size	Avg Latency	P95 Latency	SLA Status
3 Node EKS	2.10 s	2.32 s	PASS — all 3 runs under 5 s
4 Node EKS	1.75 s	2.15 s	PASS — all 3 runs under 5 s

Adding one node further reduces the mean spike latency by 17%. Both configurations meet the SLA. Performance scales predictably with additional compute capacity.

6.8 Cost Analysis

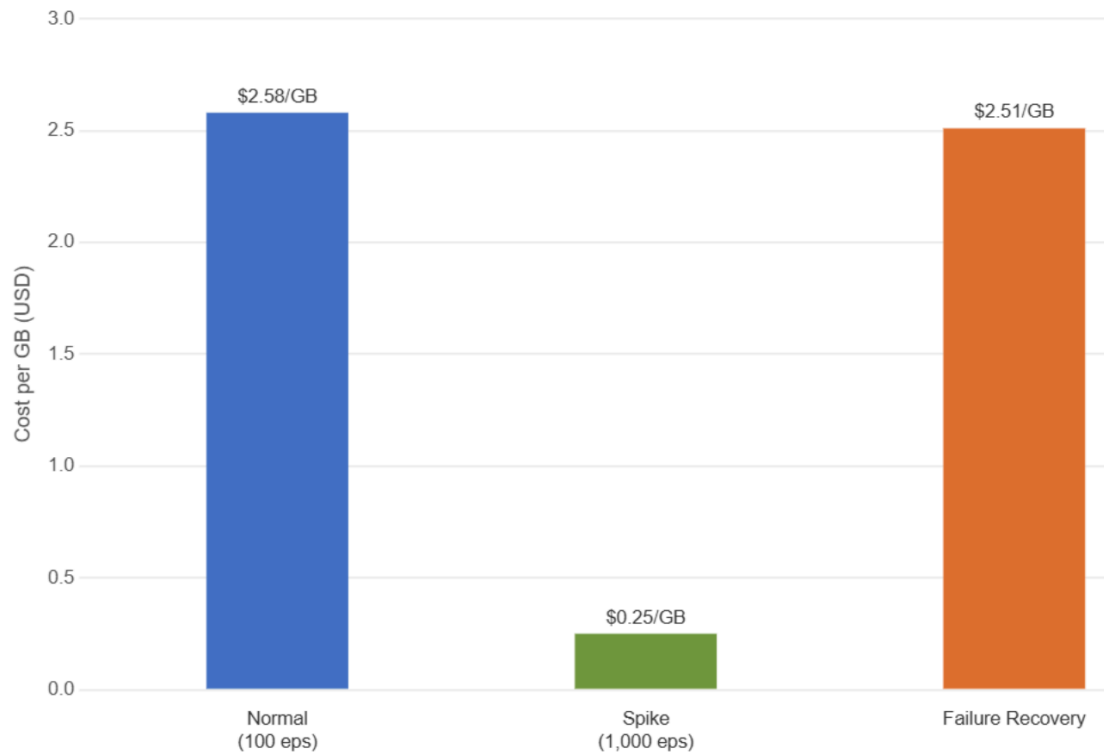


Figure 5 — Cost per GB Processed by Scenario

Scenario	Duration	Cost per Run	Data Volume	Cost per GB
Normal (100 eps, 5 min)	5 min	~\$0.017	6.6 MB	~\$2.58/GB
Spike (1,000 eps, 5 min)	5 min	~\$0.017	66.0 MB	~\$0.25/GB
Failure Recovery	10 min	~\$0.033	13.2 MB	~\$2.51/GB

The optimised pipeline achieves a 110x latency improvement under spike load with zero increase in infrastructure cost. The architectural change — separating hot and cold paths and autoscaling more efficiently utilises the compute already available. This makes the optimised pipeline strictly dominant over the baseline: lower latency, faster recovery, and equal cost per GB.

7. Research Analysis and Experimental Results

7.1 Research Direction — SLA Aware Data Processing

The project explores SLA aware data processing as its primary research and innovation contribution. This direction was selected because it directly addresses the observed root cause of the baseline pipeline's failure under spike load: the absence of any structural priority mechanism to protect latency-critical work from latency-tolerant work.

SLA-aware data processing refers to the practice of designing a streaming system so that processing responsibilities are assigned to execution paths based on their urgency. In systems where different output types have different latency requirements, this approach allows the system to protect its most critical outputs under overload by isolating them from slower, less urgent operations. This is distinct from raw performance optimisation — the goal is not simply to process events faster in aggregate, but to maintain the latency guarantee for the most operationally important output even when other parts of the system are under stress.

7.2 Motivation from Observed System Behaviour

The decision to pursue SLA aware processing was motivated directly by the baseline spike evaluation results:

Load Condition	Avg Latency	SLA Status	Observation
Normal load (100 eps)	2.67 s	PASS	System within SLA — all outputs adequate
Spike load (1,000 eps)	231.78 s	FAIL (46x over SLA)	Categorical failure — not degradation but architectural collapse

7.3 How SLA Aware Processing Helps

7.3.1 Priority Encoding at the Architecture Level

In the optimised design, the service level objective is built into the system's structure. The hot service physically cannot be slowed down by S3 writes because no S3 write code exists in the hot JVM. The cold service cannot physically skip archival work because it runs independently and maintains its own Kafka consumer group offset. Priority is enforced by architecture, not configuration. This is a meaningful distinction: a configuration-based priority system can still be violated under sufficiently heavy load because both paths compete for the same underlying resources.

7.3.2 Graceful Degradation Under Overload

Under sustained overload, the two paths degrade independently in ways that match their operational importance:

Path	Behaviour Under Overload	Impact on Operations
Hot path	May increase batch duration slightly. Redis state remains continuously updated, with no catastrophic latency blow up	Safety-critical real-time view remains operational

Path	Behaviour Under Overload	Impact on Operations
Cold path	S3 write latency may increase, breach window reports may be delayed by minutes, but this does not affect the live dashboard	Historical analysis may lag — operationally acceptable

7.4 Research Contribution

We design and evaluate an SLA aware split processing architecture for a vaccine cold chain monitoring pipeline in which the real time operational path (Redis state updates) is deployed as an independent service isolated from the archival path (S3 writes). This design ensures that overload in historical storage tasks does not compromise the latency guarantee for live breach visibility. We demonstrate empirically that this architectural choice reduces spike load mean end to end latency from 231.78 seconds to 2.10 seconds (a 110x improvement) while preserving data correctness and reducing failure recovery time from 64.4 seconds to 15.67 seconds.

7.5 Summary of Experimental Results

Scenario	Pipeline	Avg Latency	SD	P95	Recovery	SLA
Normal (100 eps)	Baseline	2.67 s	±0.07	3.19 s	—	PASS
Normal (100 eps)	Optimised	1.56 s	±0.21	1.75 s	—	PASS
Spike (1,000 eps)	Baseline	231.78 s	±5.68	231.79 s	—	FAIL
Spike (1,000 eps)	Optimised	2.10 s	±0.59	2.32 s	—	PASS
Failure Recovery	Baseline	3.27 s	±0.08	3.61 s	64.4 s	PASS
Failure Recovery	Optimised	1.78 s	±0.23	1.93 s	15.67 s	PASS
Spike (4 node)	Optimised	1.75 s	—	2.15 s	—	PASS

8. Limitations and Future Work

8.1 Current Limitations

- Synthetic workload scope: The evaluation workload covers 50 simulated devices across 10 facilities with a fixed invalid rate of 1.82%. Real deployments would exhibit higher variability in device quality, network reliability, and temporal event distribution.
- Single AWS region: All experiments were conducted in ap south 1. Multi region deployments would introduce cross region replication latency, which the current architecture does not address.
- Static facility lookup: The device to facility mapping is loaded as a static broadcast variable at Spark application startup. A production system would require a dynamic lookup service.

8.2 Future Work

Priorit y	Enhancement	Expected Benefit
1	Serverless cold path	Event driven S3 Lambda instead of a continuous pod eliminates idle cold path compute cost during off-peak hours
2	Schema registry integration	Confluent Schema Registry for independent schema evolution and centralised audit trail for schema changes
3	Multi facility threshold updates	Secondary Kafka topic for threshold change events allow adjustment without pipeline restart
4	Geographic aggregation dashboard	Grafana panel with location_lat/lon overlay for district health officers to identify spatial clusters of temperature breaches
5	Elastic Auto Scaling	Improved scalability and performance with automatic handling of workload spikes, reduced resource wastage through efficient scaling, and lower operational overhead due to minimal manual intervention.

9. Conclusion

The VacciGuard project demonstrates that meeting a real-time latency SLA in a vaccine cold chain monitoring pipeline is not primarily a matter of raw compute power, but of architectural priority. The baseline pipeline was computationally capable of processing 1,000 events per second — it did so correctly across all three runs — but lacked a mechanism to protect the latency-sensitive Redis path from the slower archival path.

The SLA-aware split pipeline resolves this by making the service-level objective an explicit structural concern. The hot service is narrow, fast, and uncapped; the cold service is thorough and permitted to be slow. The empirical results clearly support this conclusion. The optimised pipeline reduced the mean spike load latency from 231.78 seconds to 2.10 seconds (a 110x improvement) while preserving throughput, data quality, and correctness across all three experimental scenarios.

The broader lesson applies beyond vaccine cold chain monitoring: in any streaming system that produces multiple output types with different urgency profiles, treating all work as structurally equivalent is an architectural risk that will manifest under load. SLA aware decomposition is the appropriate response, and this project provides a concrete, evaluated instance of that design pattern applied to a safety-critical healthcare monitoring context.

Appendix : Technology Stack Reference

Layer	Technology	Version / Configuration
Message broker	Apache Kafka	2.5.0 8 partitions per topic
Stream processing	Apache Spark Structured Streaming	PySpark 3.x checkpoint-based recovery
Hot storage	Amazon ElastiCache (Redis)	OSS compatible latest state per device_id
Cold storage	Amazon S3	Parquet + JSON output formats
Container orchestration	Amazon EKS	Kubernetes 1.x ap south 1
Infrastructure as Code	Terraform	AWS provider
Metrics collection	Prometheus	Scrape interval: 5 s Kubernetes SD
Visualisation	Grafana	11.1.0 self managed on EKS
Workflow orchestration	Apache Airflow	Daily compliance report DAG
Cloud region	AWS ap south 1	Mumbai

References

[Tanish520/vacciguard: Cloud-native vaccine cold chain monitoring pipeline — CSG527](#)

Operational Screenshots

```

VacciGuard Pipeline Demo
Failure Injection & Recovery · Amazon EKS

[demo] Plan: 60s normal load → pod kill → live recovery
[demo] Total workload duration: 4 min

Pre-flight checks

[demo] Exporting AWS credentials...
[demo] Verifying AWS identity...
[demo] AWS credentials valid.
[demo] Verifying kubectl cluster connection...
[demo] kubectl connected: vacciguard-tanish-baseline-eks
[demo] Checking for stale evaluation jobs...
[demo] No stale jobs.
[demo] Checking monitoring stack...
[demo] Monitoring stack is healthy.
[demo] Starting Grafana port-forward...
[demo] Grafana ready → http://127.0.0.1:58344

Dashboard URL:
http://127.0.0.1:58344/d/vacciguard-pipeline-overview/vacciguard-live-demo-wallboard?orgId=1&refresh=5s&from=now-10m&to=now

Dashboard is open. Position your screen for recording.
Press Enter when you are ready to start.

```

```

PHASE 1 - Normal Load (100 eps - 60s)

[demo] What to show on dashboard:
[demo]   • Ingest Rate      ~100 events/s
[demo]   • Consumer Lag     0
[demo]   • Avg Latency      = 0.8 s (well under 5 s SLA)
[demo]   • SLA Status       GREEN

[demo] Starting evaluation in background (run_id: demo-failure-recovery-20260419T120933Z)...
[demo] Evaluation running (pid 34063). Logs - /Users/tanishgupta/Documents/GitHub/vacciguard/.worktrees/baseline-spike-fix/artifacts/monitoring/eval-20260419T120933Z.log
[demo] Waiting 60s for normal load to be visible on the dashboard...

[demo] Normal load running - 60s remaining
[demo] Normal load running - 50s remaining
[demo] Normal load running - 40s remaining
[demo] Normal load running - 30s remaining
[demo] Normal load running - 20s remaining
[demo] Normal load running - 10s remaining

```

PHASE 2 - Pod Failure Injection

►► INJECTING FAILURE: deleting pod stream-processor-hot-b54b8dbf6-r47hx

pod "stream-processor-hot-b54b8dbf6-r47hx" deleted from vacciguard namespace

[demo] Pod deleted. Kubernetes will schedule a replacement automatically.

[demo] Watch the dashboard:

[demo] • Pod Restart Count → increments by 1
[demo] • Queries Active → drops to 0 briefly
[demo] • Ingest Rate → dips, then recovers
[demo] • SLA Status → returns GREEN within ~15 s

PHASE 3 - Recovery

```

[demo] Waiting for stream-processor-hot to come back Running...
[demo] New pod is Running: stream-processor-hot-b54b8dbf6-r47hx
[demo] Recovery complete - SLA should be GREEN on the dashboard.

[demo] Waiting for evaluation controller to finish and upload report to S3...
[demo] Evaluation completed successfully.

[demo] Fetching evaluation report from S3...
[demo] Report not yet available at: s3://vacciguard-tanish-baseline-ap-south-1-data/evaluations/optimized/failure-recovery/demo-failure-recovery-20260419T120933Z/report.md
[demo] The evaluation may still be processing. Check S3 in a minute.

```

Demo complete.

Dashboard (15-min view): <http://127.0.0.1:58344/d/vacciguard-pipeline-overview/vacciguard-live-demo-wallboard?orgId=1&refresh=5s&from=now-15m&to=now>

S3 report: <s3://vacciguard-tanish-baseline-ap-south-1-data/evaluations/optimized/failure-recovery/demo-failure-recovery-20260419T120933Z/>

Stop tunnel when done: `bash scripts/stop-grafana-port-forward.sh`

```

## Metrics
| Metric | Baseline |
|---|---:|
| Workload family version | evaluation-workload-v1 |
| Scenario | failure-recovery |
| Configured replay rate | 100.00 events/s |
| Stream metrics source | metrics_endpoint |
| Avg end-to-end latency | 1.33 s |
| P95 latency | 1.74 s |
| P99 latency | 1.75 s |
| Throughput | 100.00 events/s |
| Input events | 26400 |
| Processed events | 25834 |
| Cumulative processed events | 25834 |
| Processed rate | 97.86% |
| Invalid events | 480 |
| Invalid rate | 1.82% |
| Deduplicated events | 86 |
| Deduplication rate | 0.33% |
| Breach events | 402 |
| Pod restart count | 2.00 |
| Queries active | 1.00 |
| Hot batch duration | 0.96 s |
| Cold batch duration | 12.28 s |

```

Grafana Dashboard

